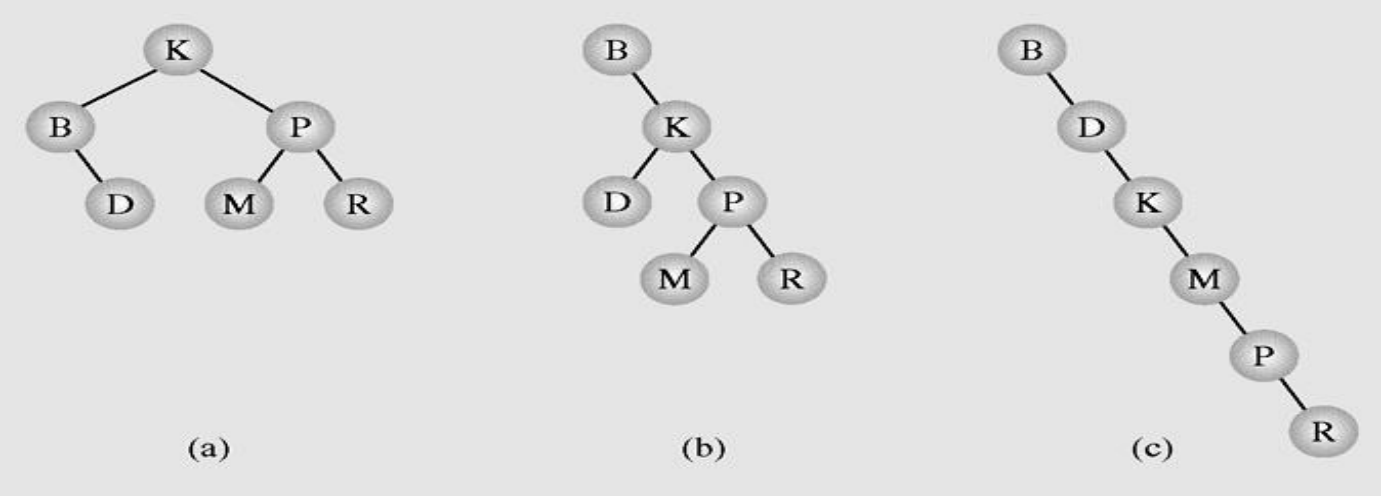
Balanced Trees

Introduction

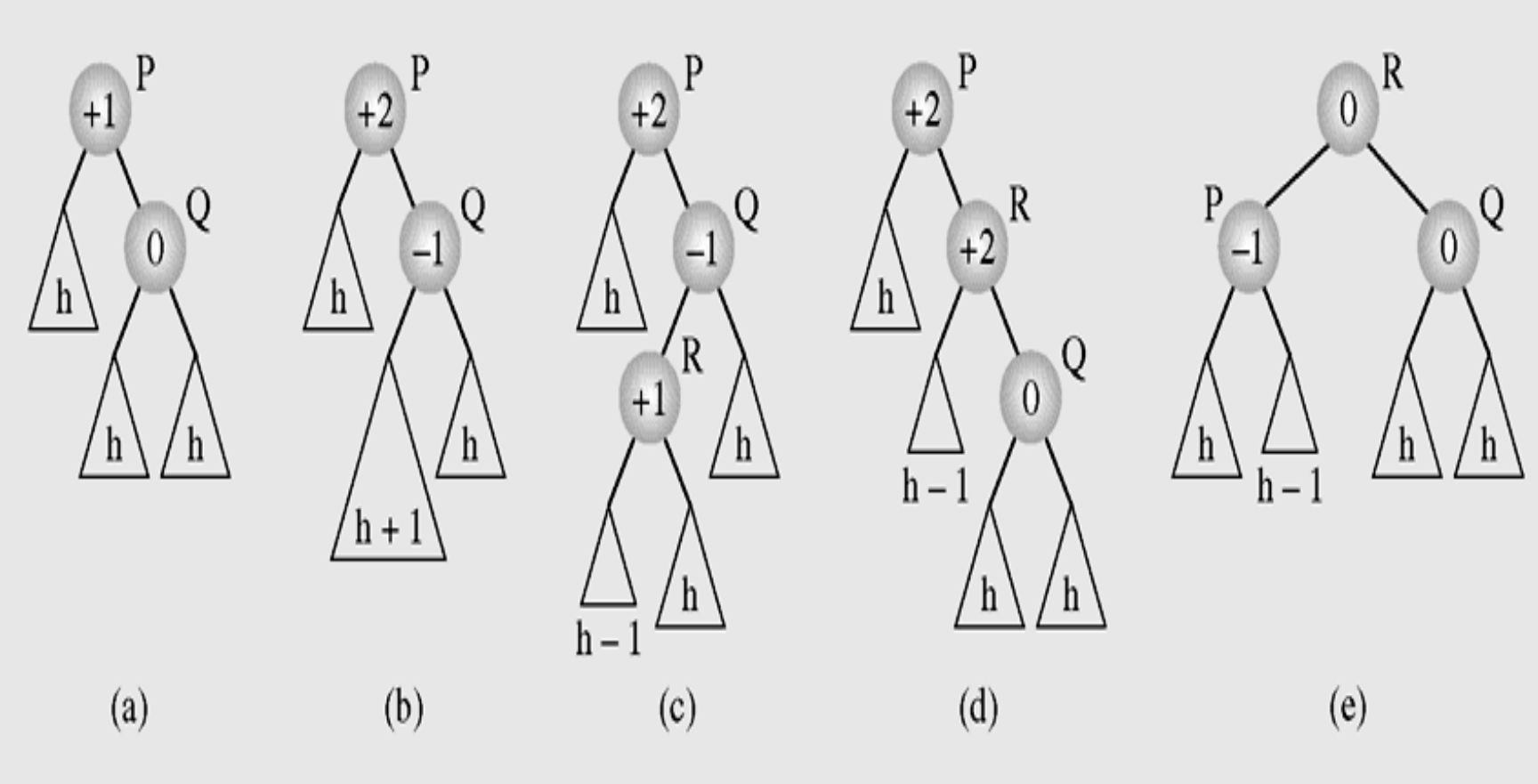
- Different BST with the same information



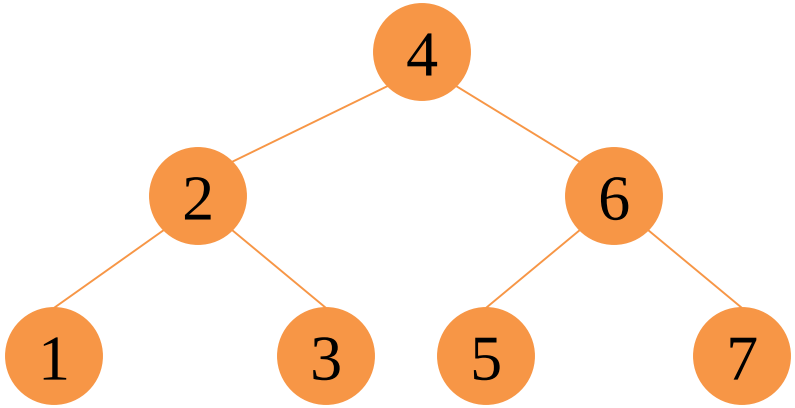
- Balancing a Tree
  - A binary tree is height-balanced or balanced if the difference in height of both sub-trees of any node in the tree is either **zero** or **one**

Balance Factor

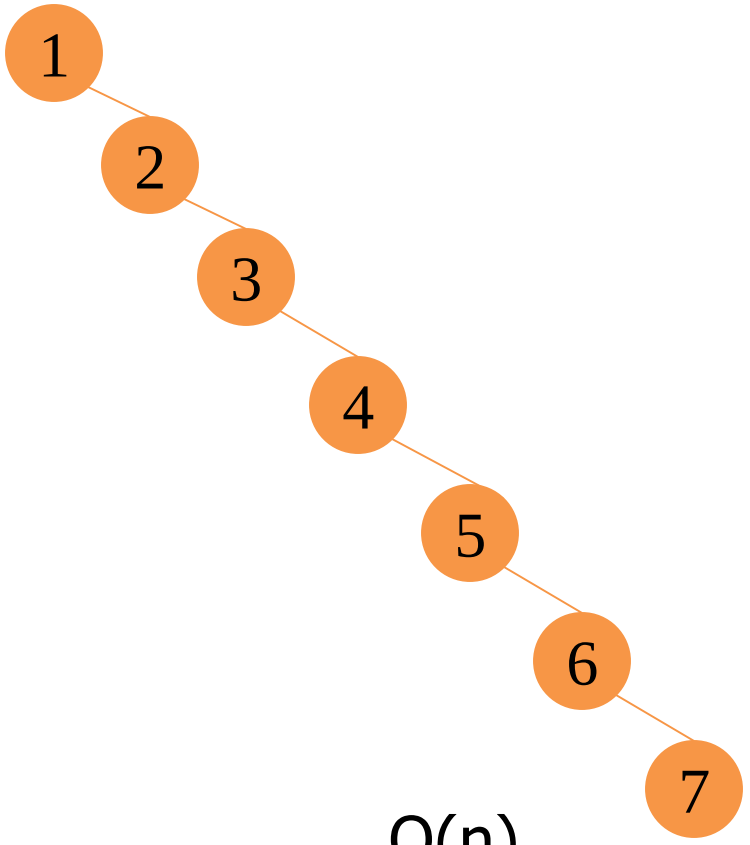
Balance factor = height(right subtree) - height(left subtree)



Balanced and unbalanced BST



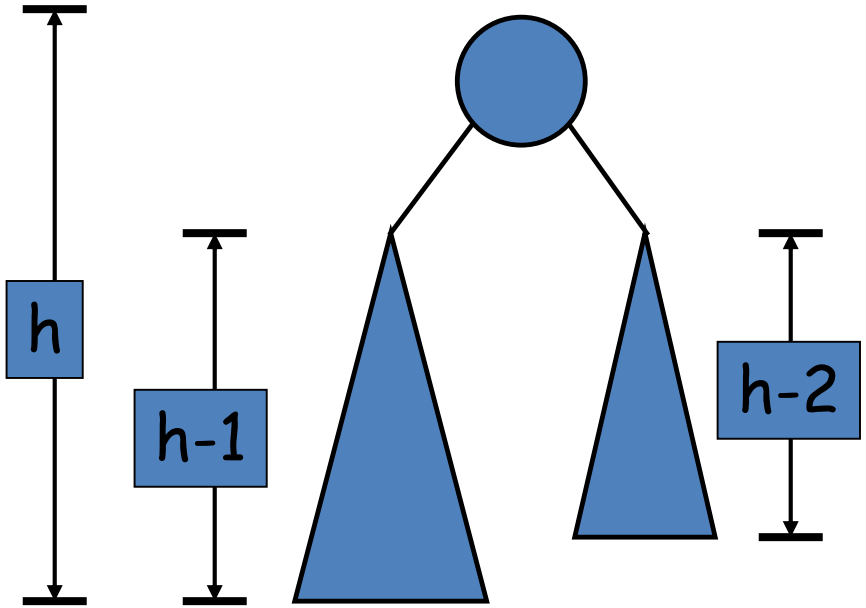
O(log n)



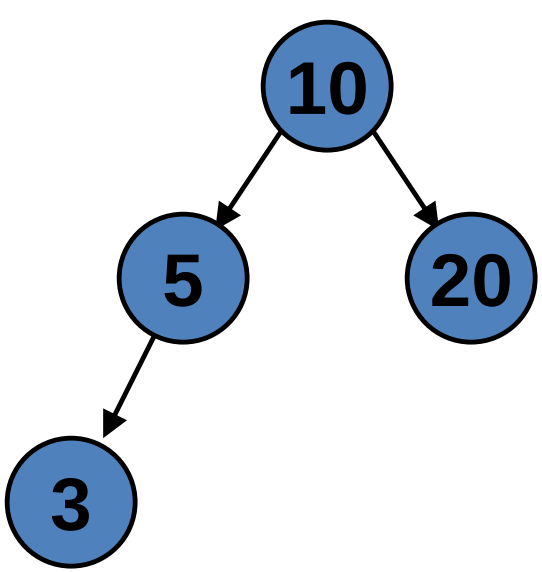
O(n)

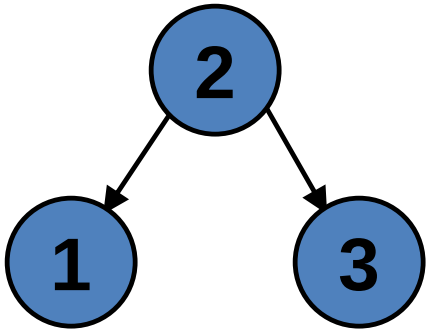
AVL (Adelson-Velskii and Landis) Trees

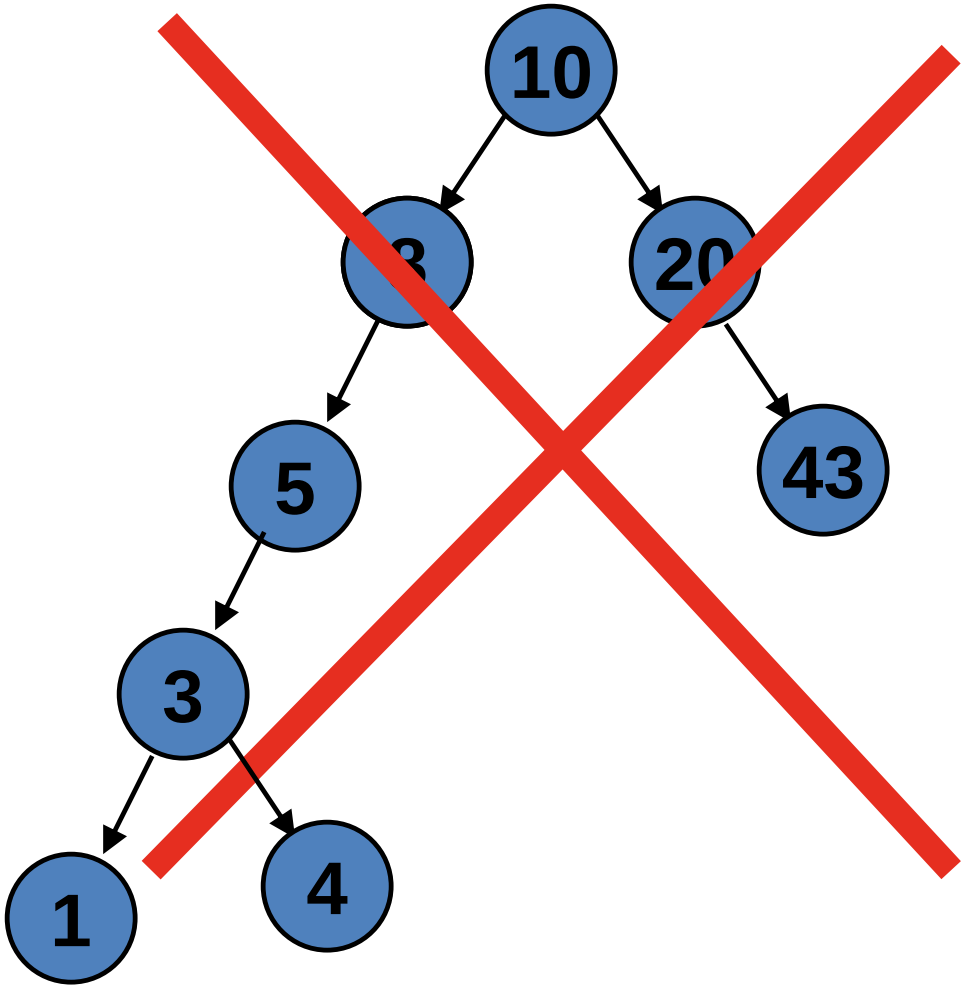
- AVL trees maintain balance.
- For each node in the tree, height of left subtree and right subtree differ by at most 1



AVL Trees

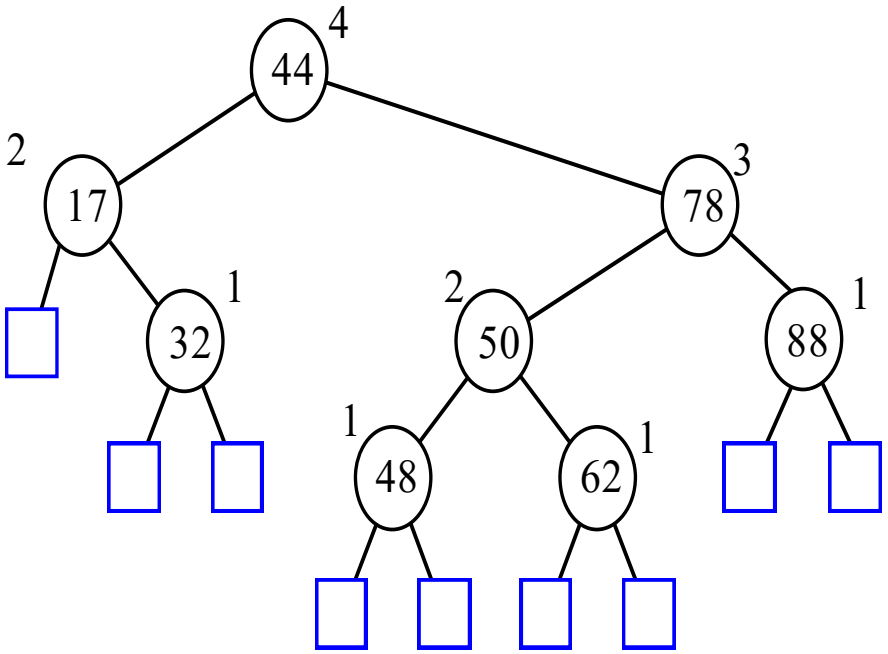






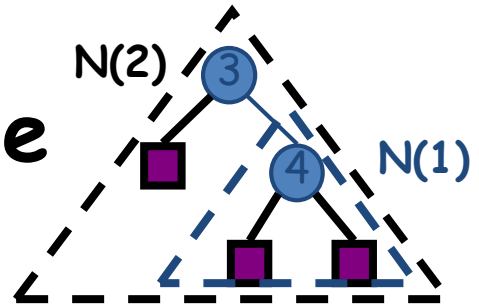
AVL Tree

An AVL Tree is a *binary search tree*, where for every internal node v of T, the *heights of the children of v differ by at most 1*.



AVL tree where the heights are shown next to the nodes

Height of an AVL Tree



- **Fact**: The ***height*** of an AVL tree storing n keys is O(log n).

- **Proof**: Let **N(h):** the minimum number of nodes in an AVL tree of height h.
  - Base case: N(0) = 1 and N(1) = 2
  - For n > 2, an AVL tree of height h contains the root node, one AVL subtree of height h-1 and another of height h-2.
  - That is, N(h) = 1 + N(h-1) + N(h-2)
  - Now, N(h-1) > N(h-2), we get N(h) ≥ 2N(h-2). Therefore,
    N(h) ≥ 2N(h-2) = 4N(h-4) = 8N(n-6) … (by substitution),
    N(h) ≥ 2^iN(h-2i)

  - Let h-2i=0 => i = h/2 => N(h) ≥ $2^{h/2}$ using base case N(0)=1
  - If AVL tree of height h stores n nodes, then n ≥ $2^{h/2}$ => h = O(log n)

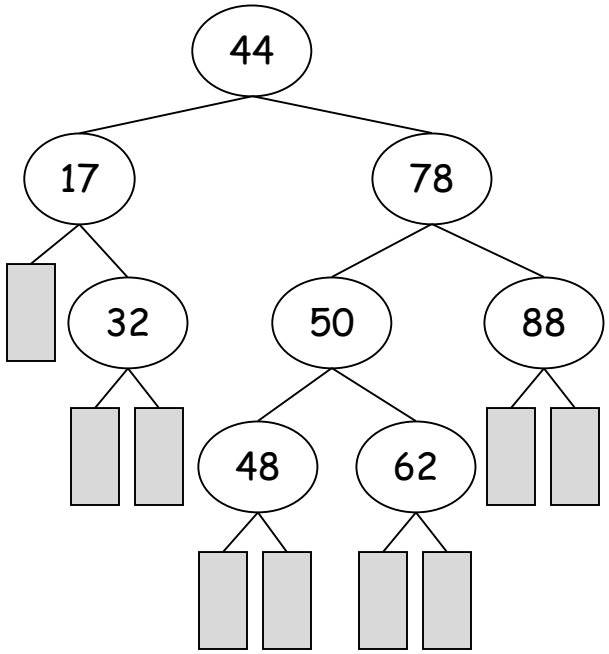
Insertion operation for AVL Tree

- Insertion is done as in a binary search tree. Always done by expanding an external node.

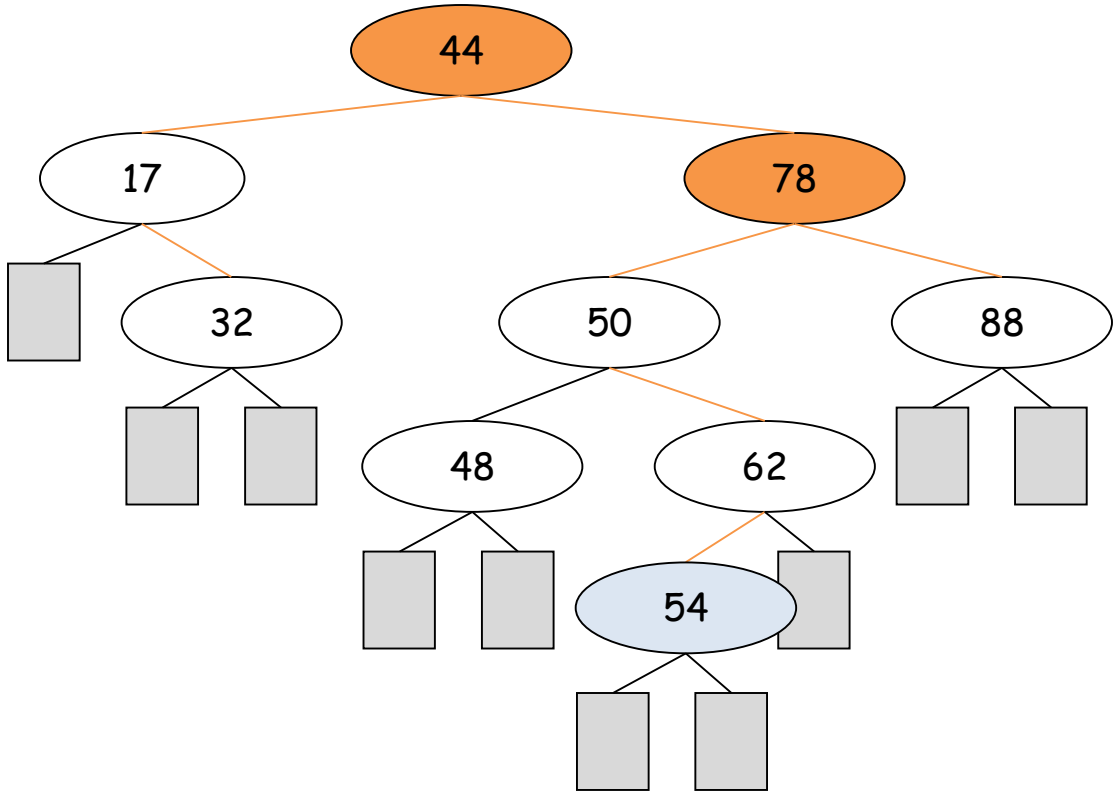
- After insertion, the balance condition may not hold in certain node(s). If so, we have to perform one of the following:

  - Single rotation
  - Double rotation

Insertion example

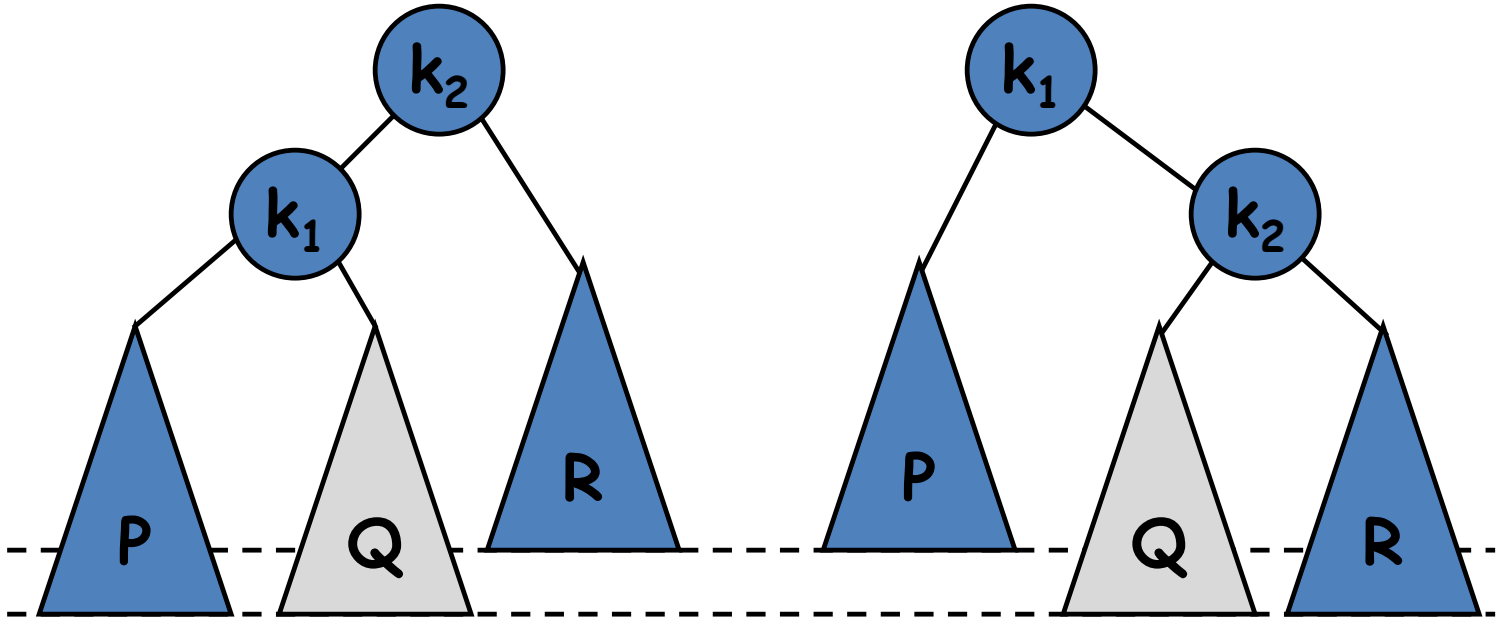


before insertion



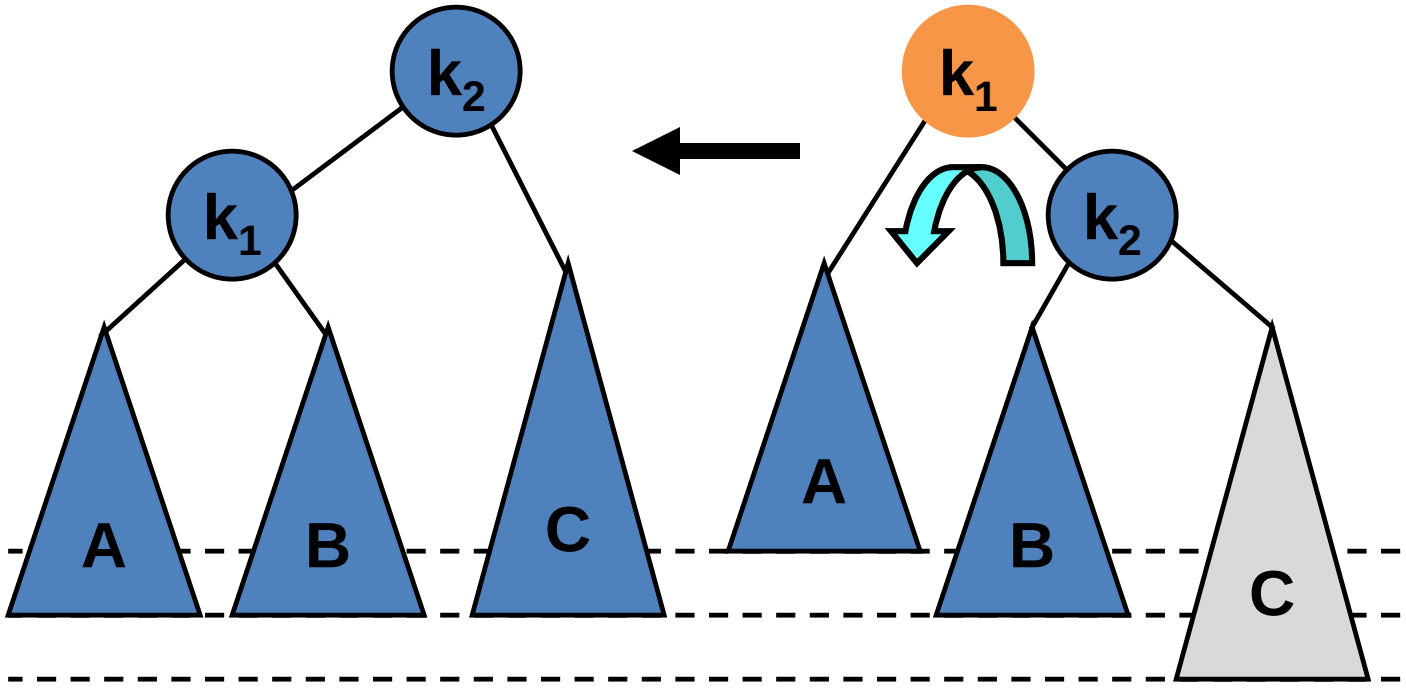
after insertion

Insertion may cause imbalance



- An insertion into the subtree:
  - P (outside) - case 1
  - Q (inside) - case 2
- An insertion into the subtree:
  - Q (inside) - case 3
  - R (outside) - case 4

Example of Single Rotation (case 4)



Insertions for AVL Trees

Let the node that needs rebalancing be α.

There are 4 cases:

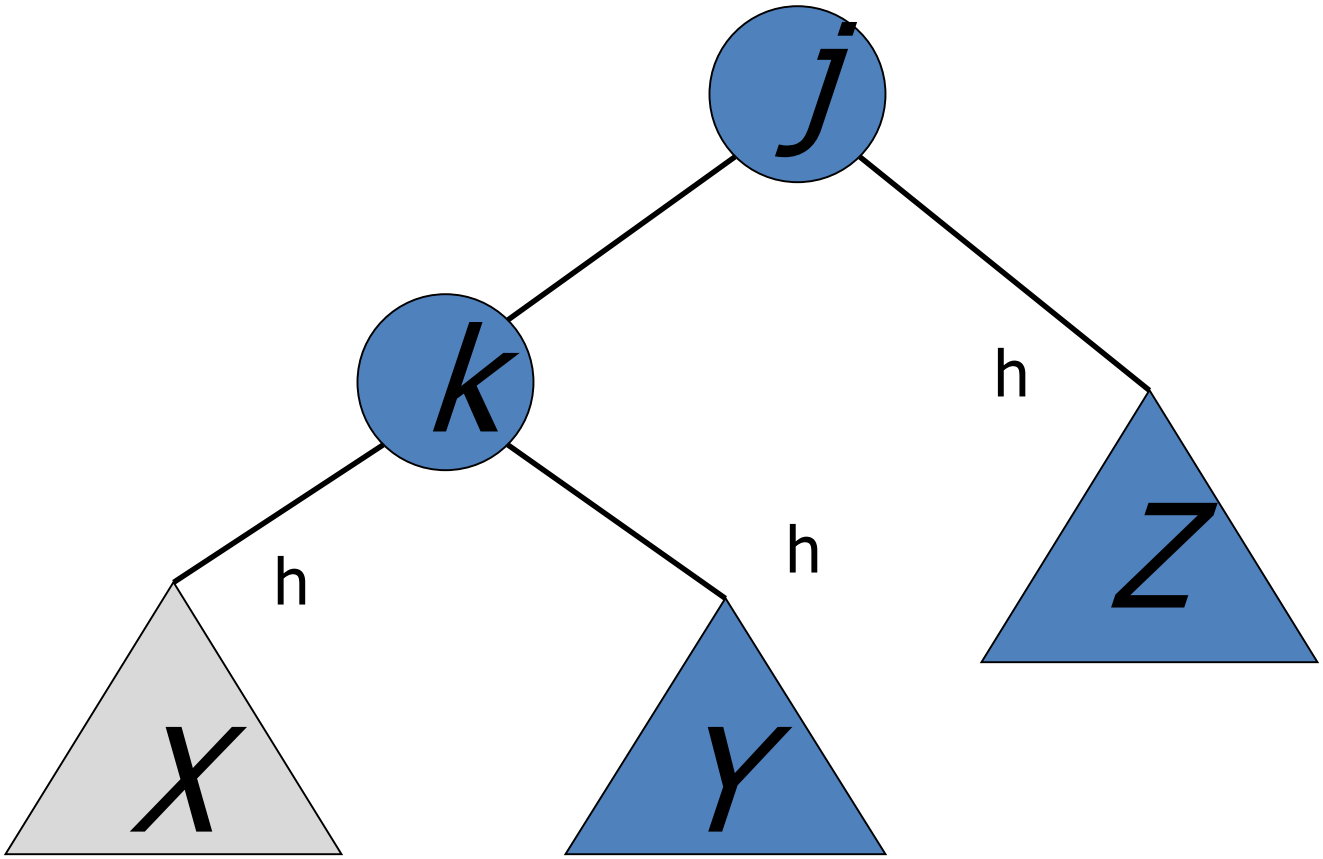
Outside Cases (require **single** rotation) :

- Insertion into left subtree of left child of α.
- Insertion into right subtree of right child of α.

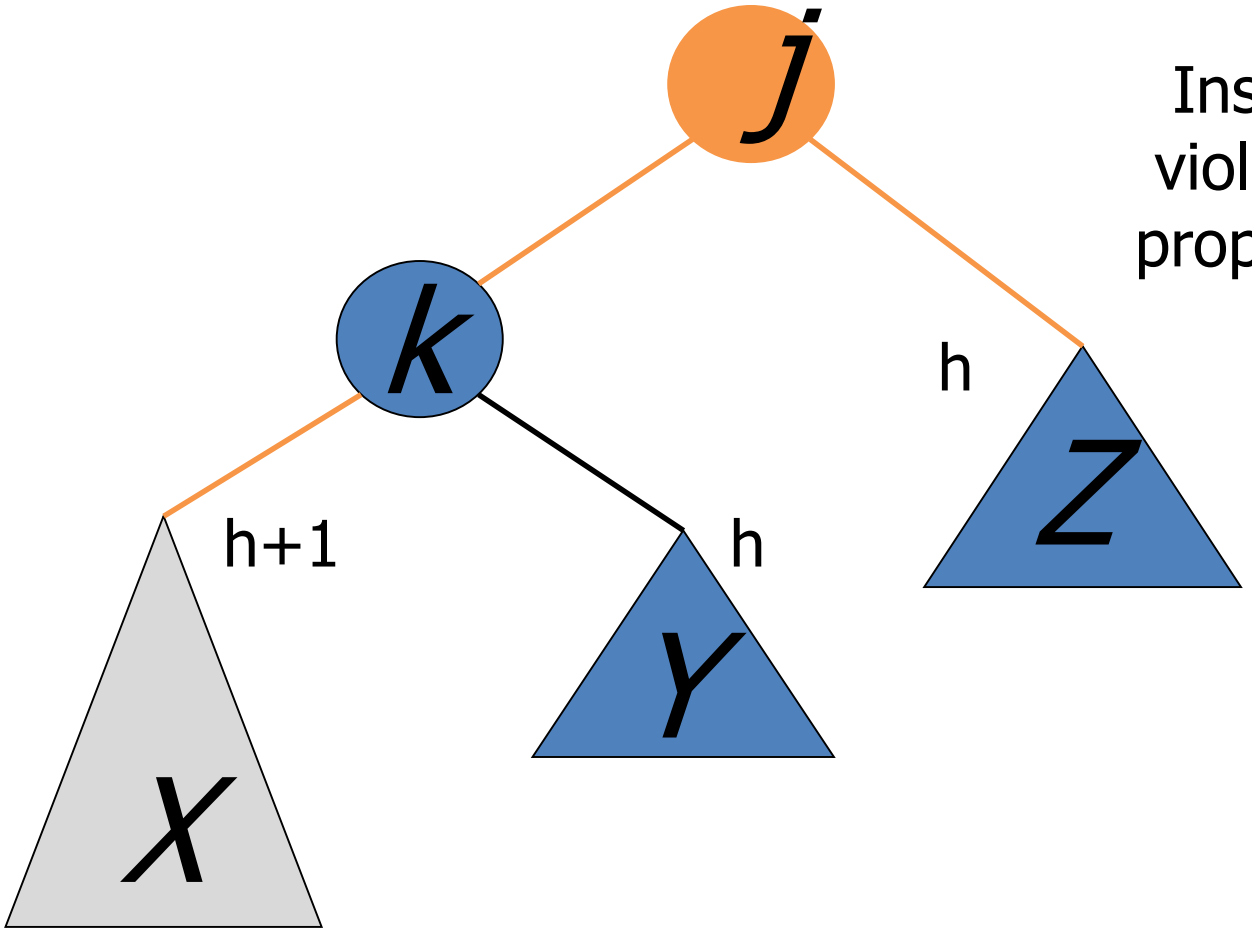
Inside Cases (require **double** rotation) :

- Insertion into right subtree of left child of α.
- Insertion into left subtree of right child of α.

AVL Insertion: Outside Case

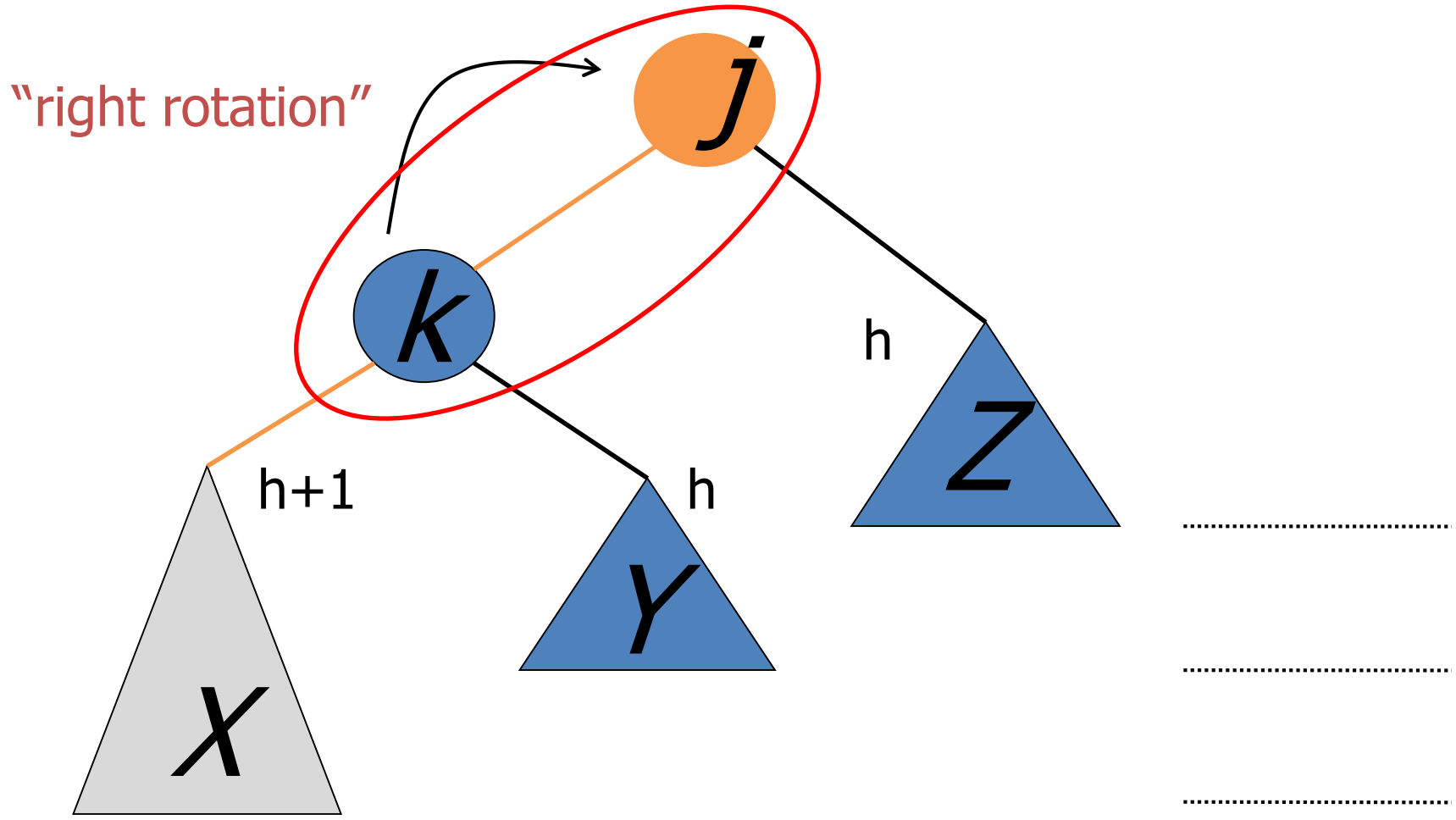


AVL Insertion: Outside Case

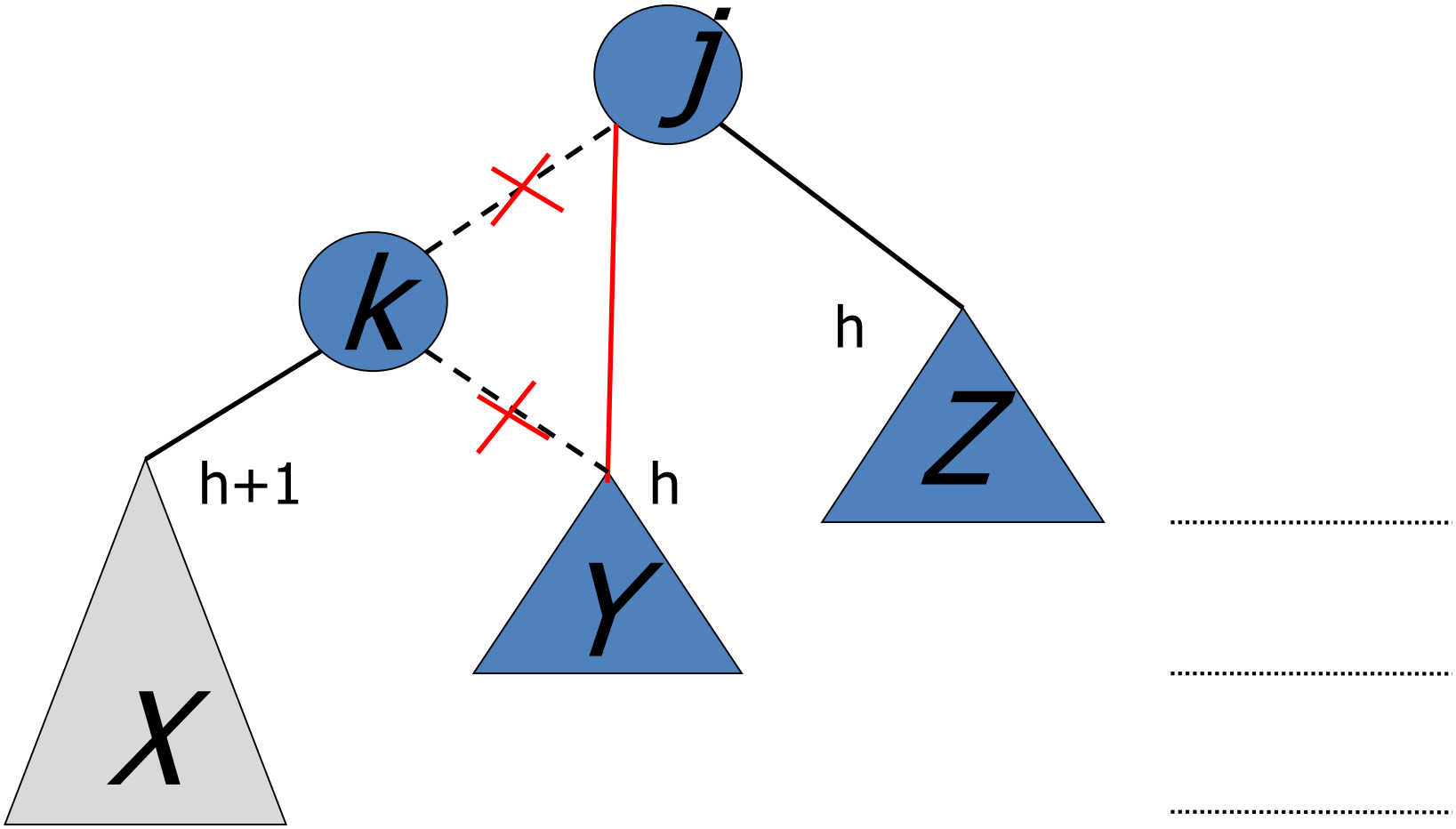


Inserting into X violates the AVL property at node j

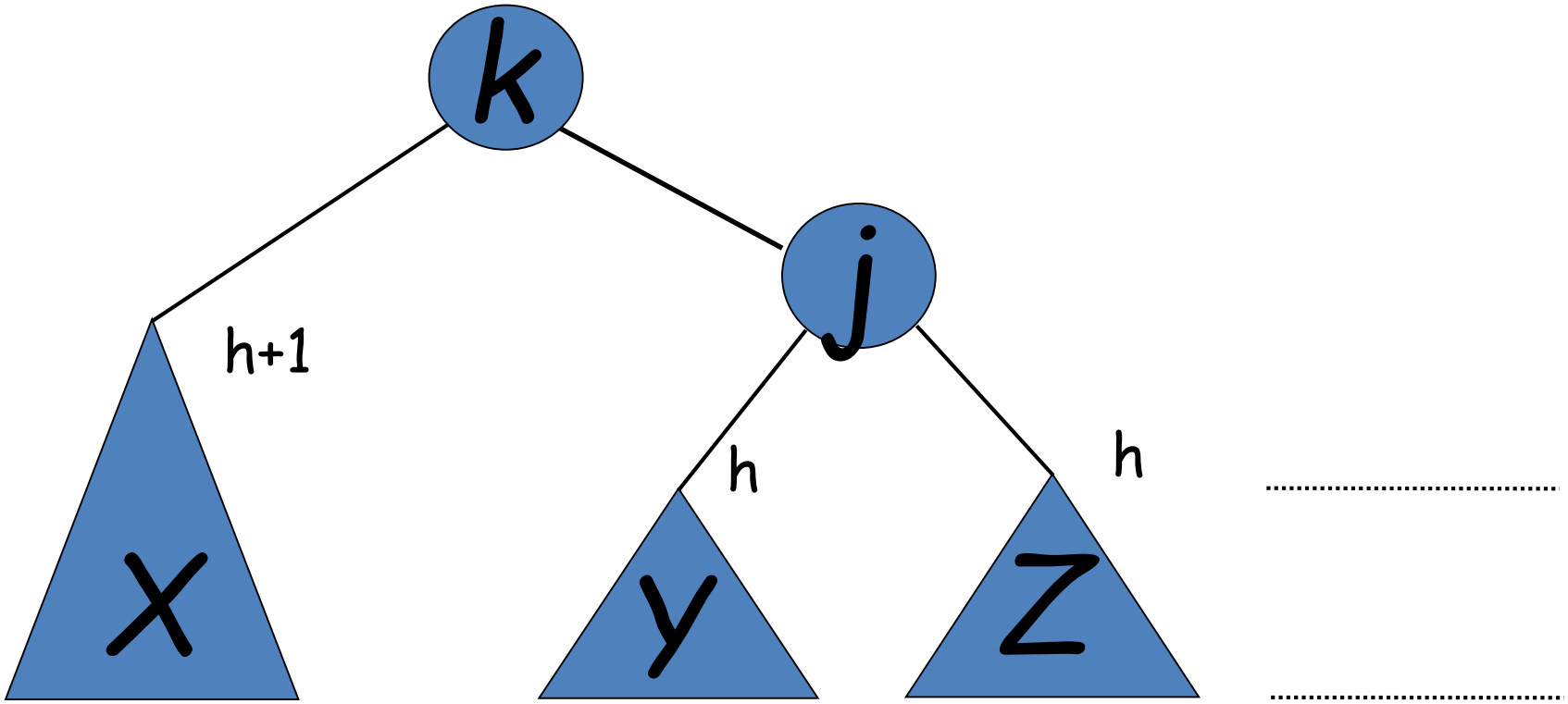
AVL Insertion: Outside Case



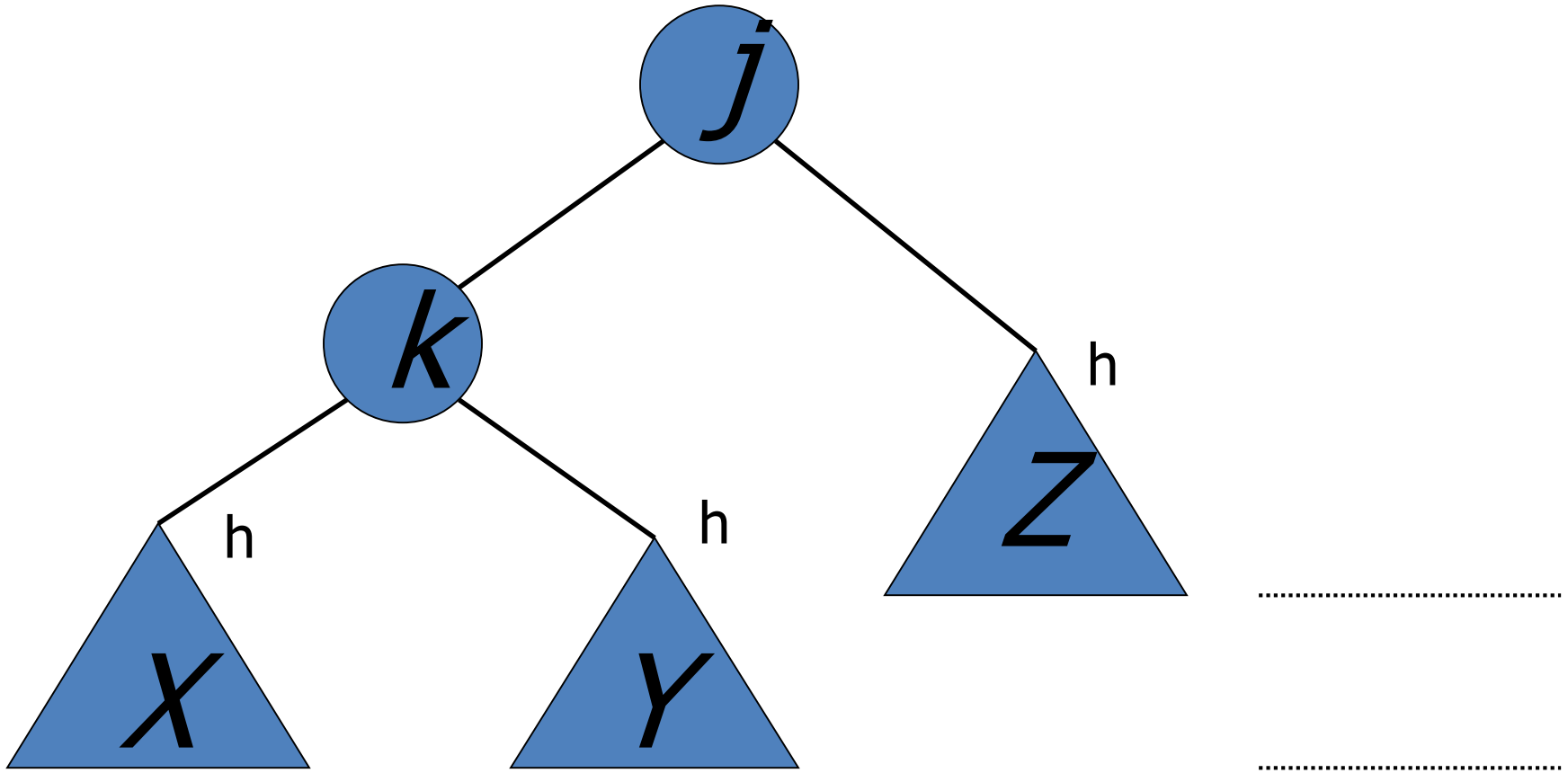
Single right rotation



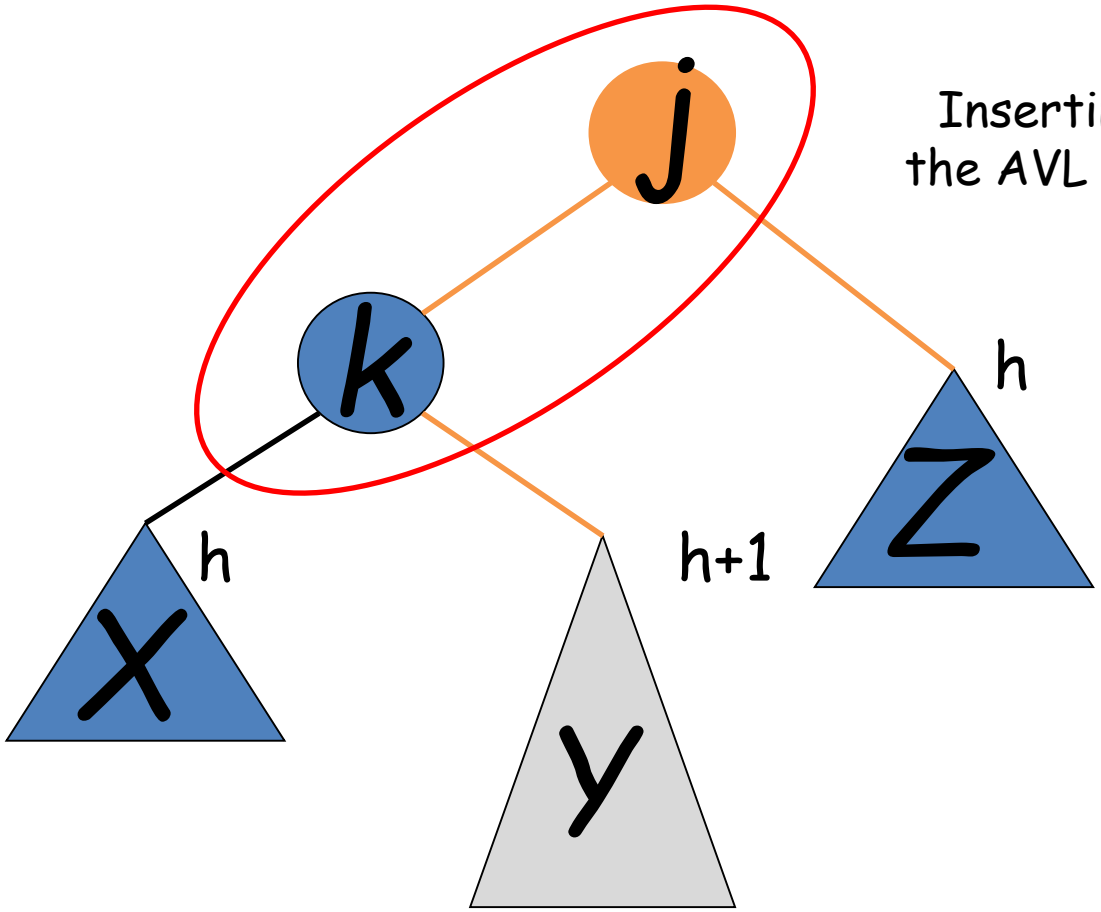
AVL property has been restored!



AVL Insertion: Inside Case



AVL Insertion: Inside Case



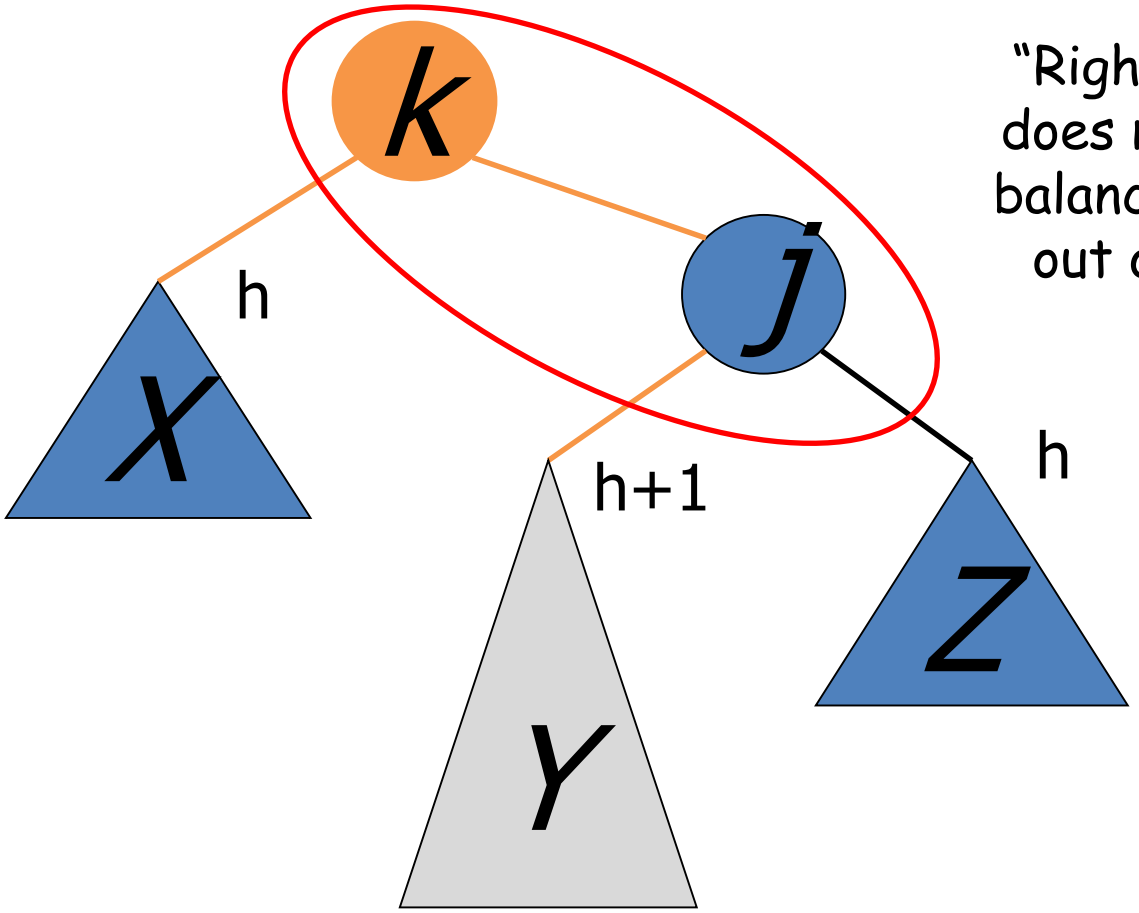
Inserting into Y violates the AVL property at node j

.....

.....

.....

AVL Insertion: Inside Case



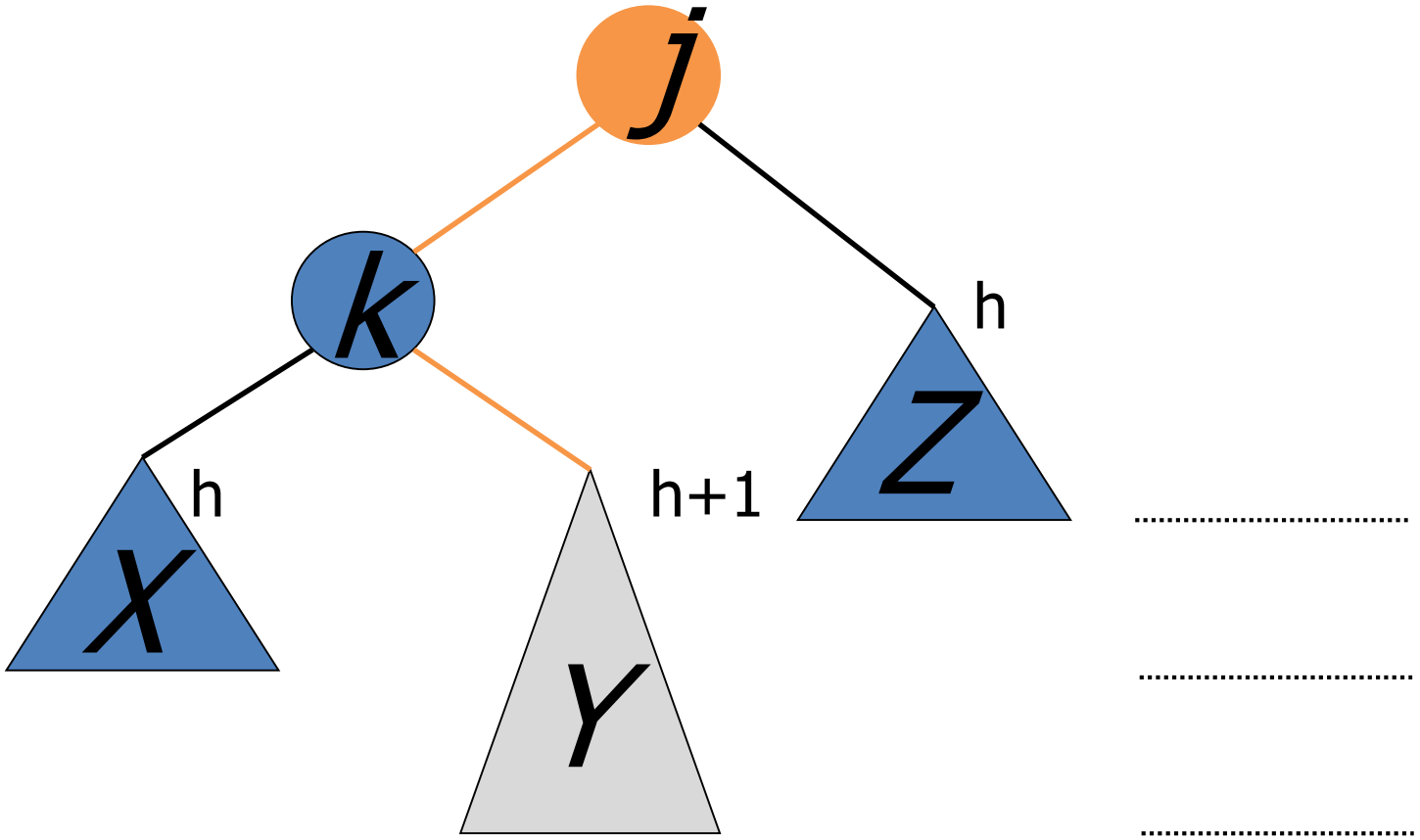
"Right rotation" does not restore balance… now k is out of balance

.....

.....

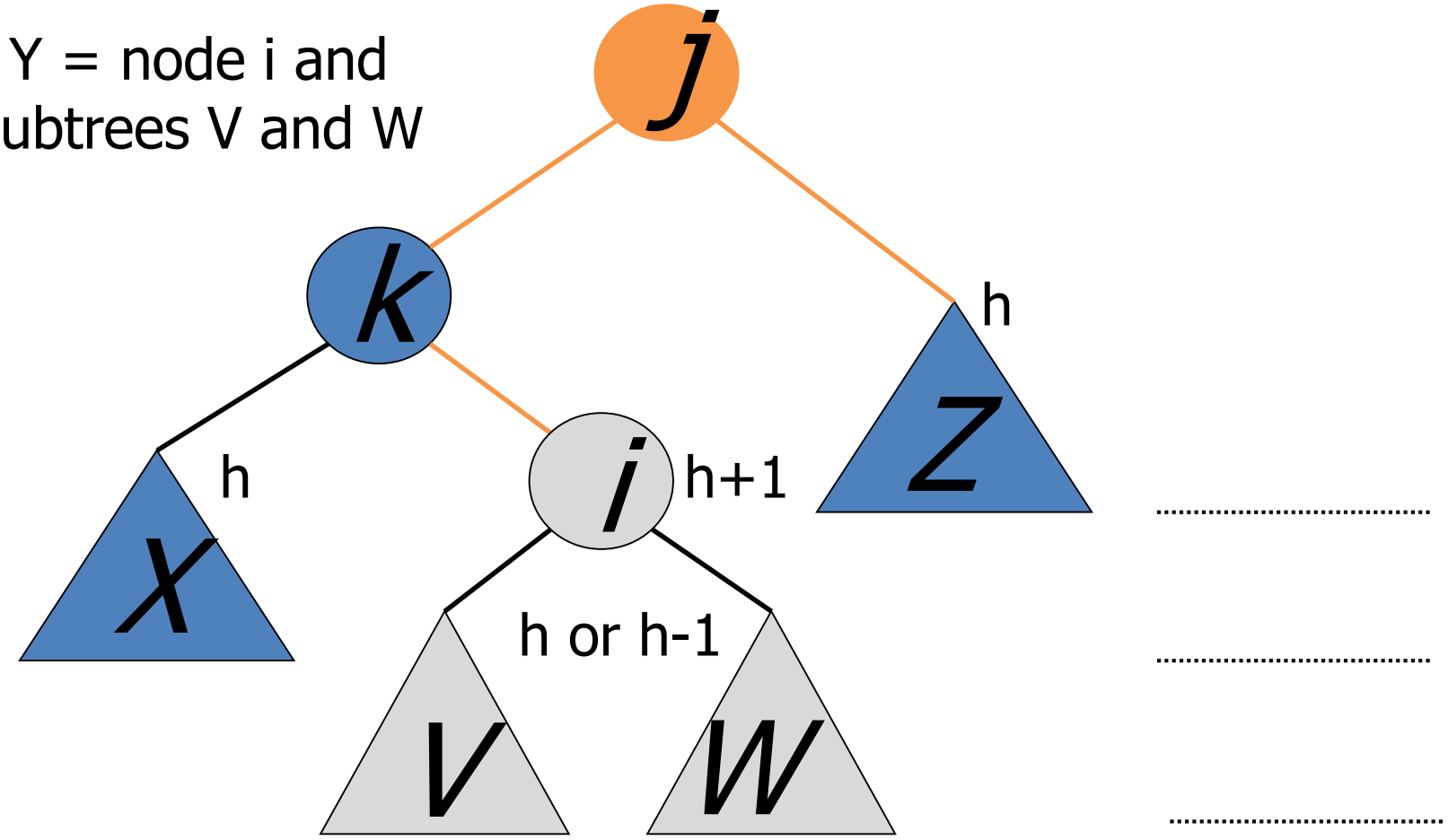
.....

AVL Insertion: Inside Case



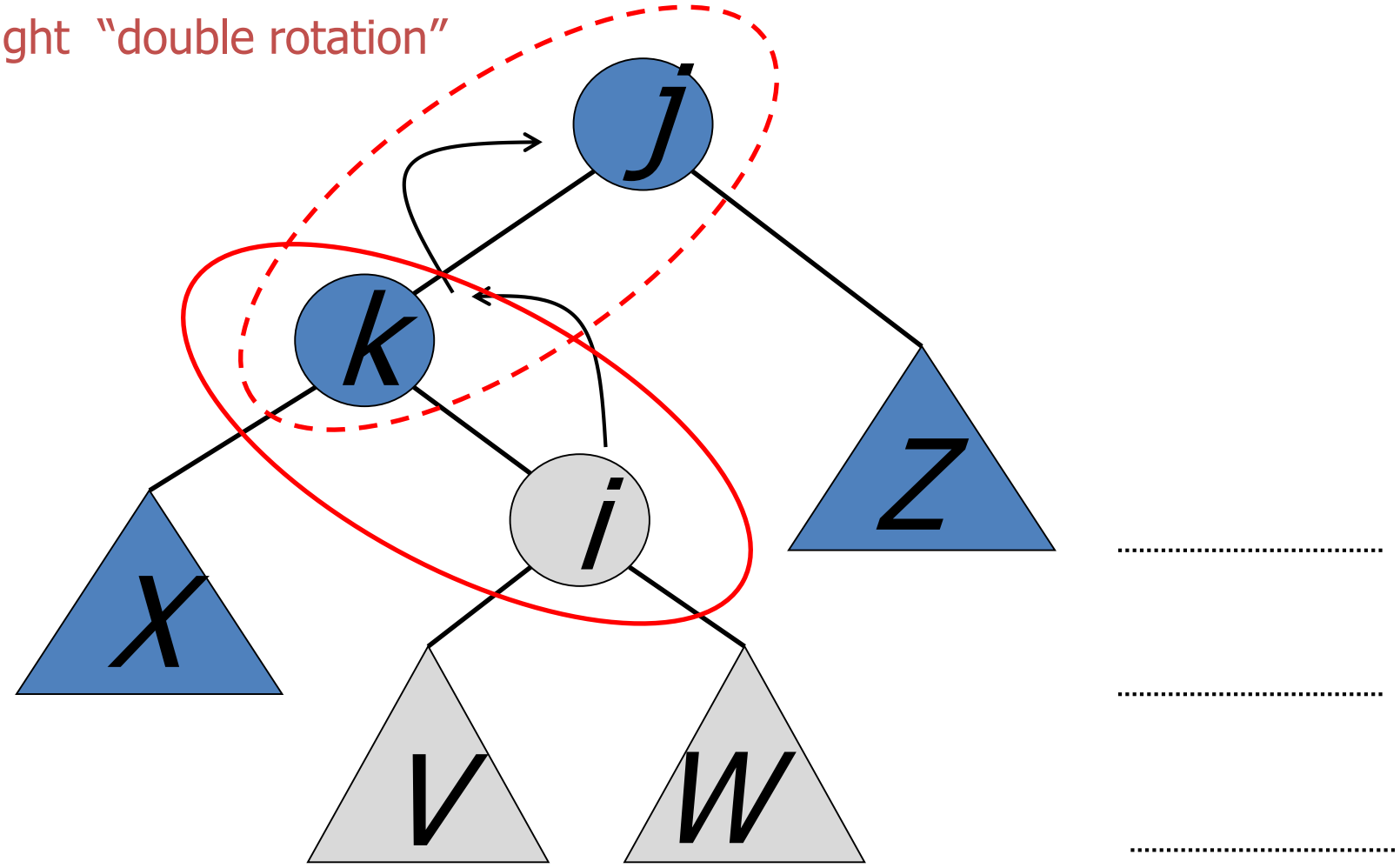
AVL Insertion: Inside Case

Y = node i and
subtrees V and W

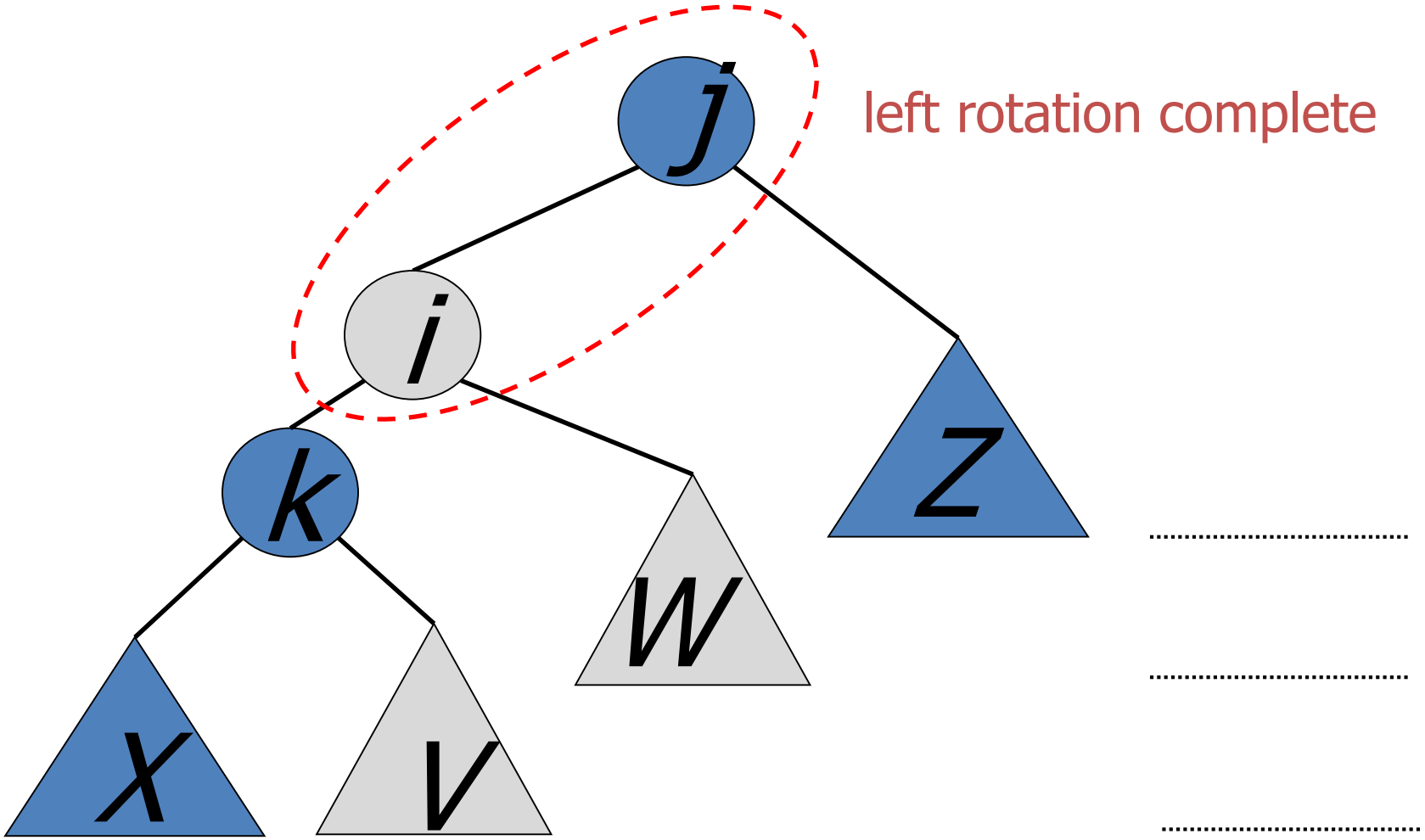


AVL Insertion: Inside Case

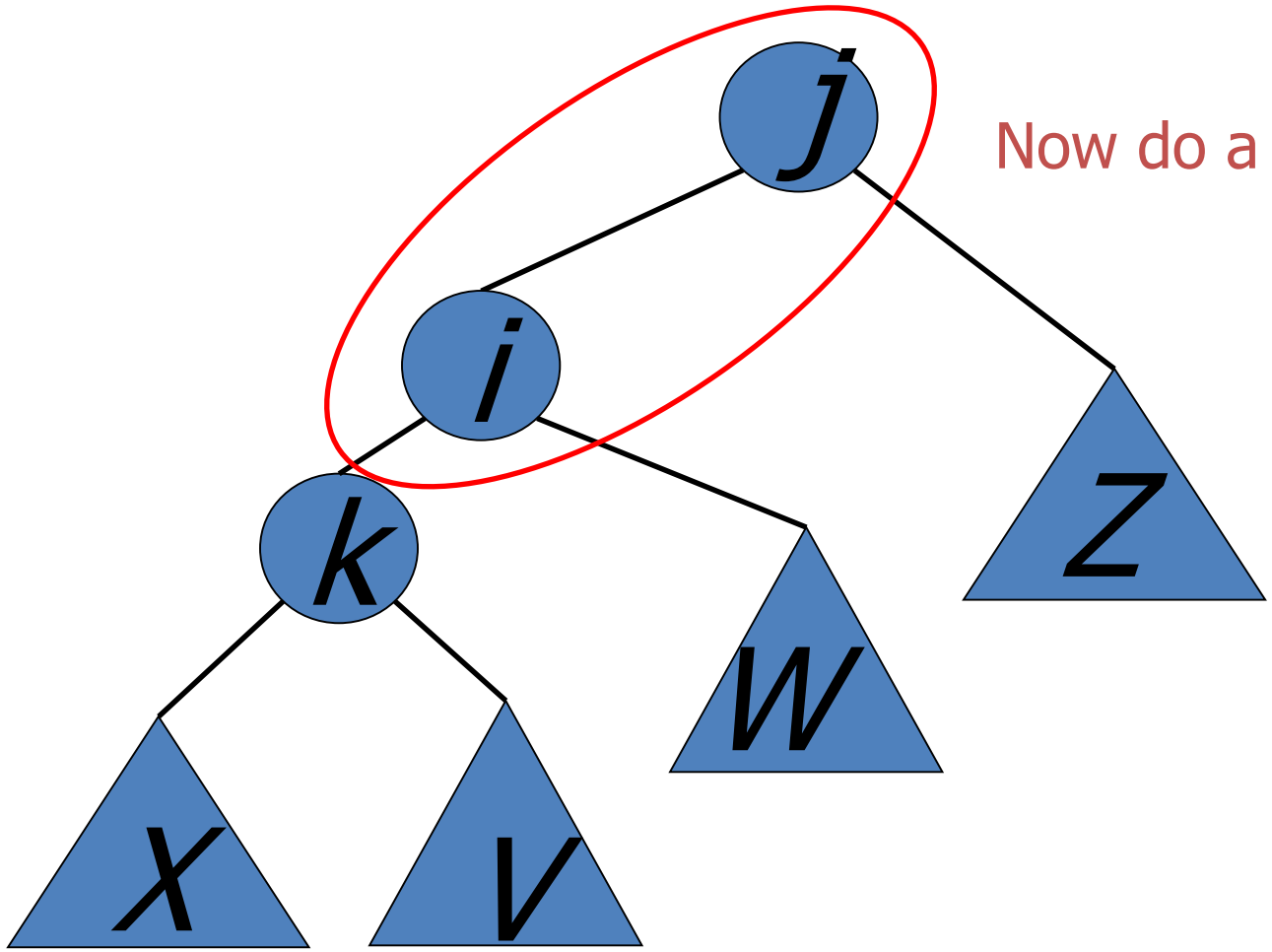
left-right "double rotation"



Double rotation : first rotation



Double rotation : second rotation



Now do a right rotation

.....

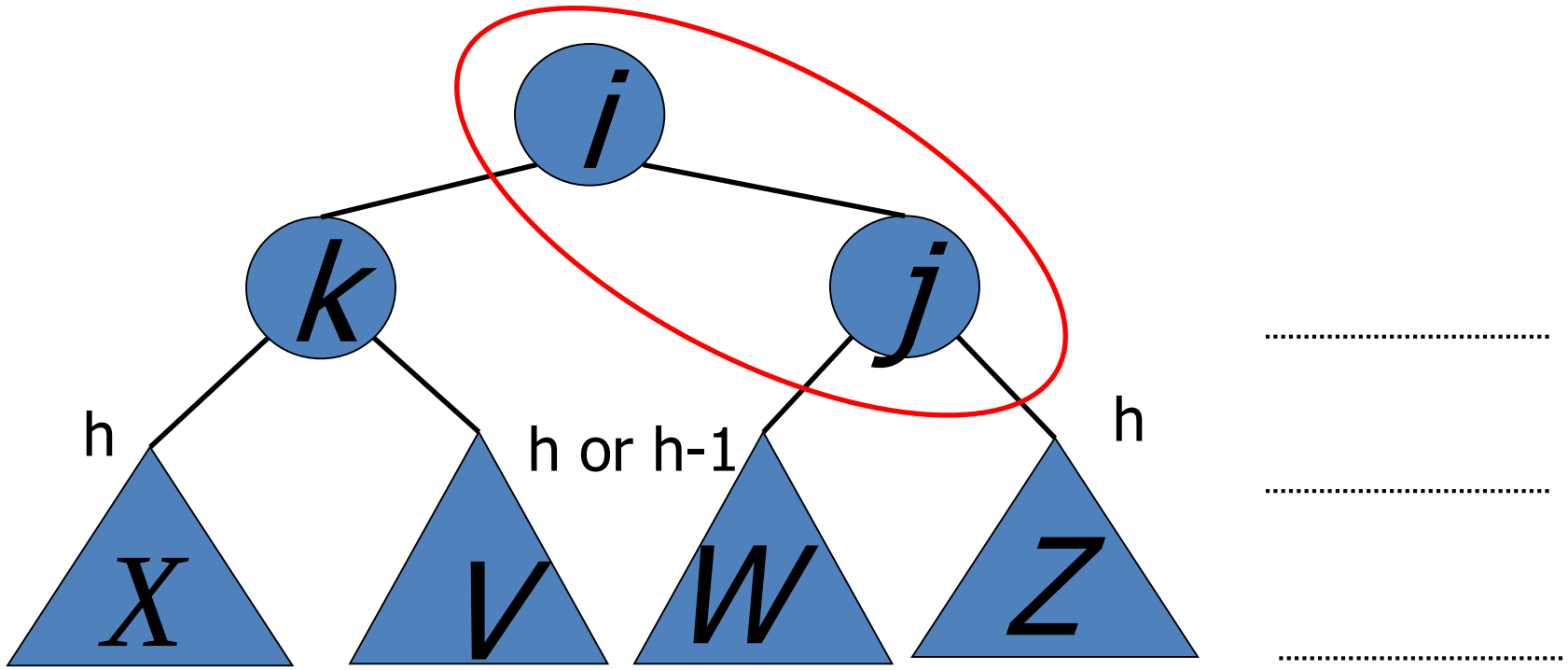
.....

.....

Double rotation : second rotation

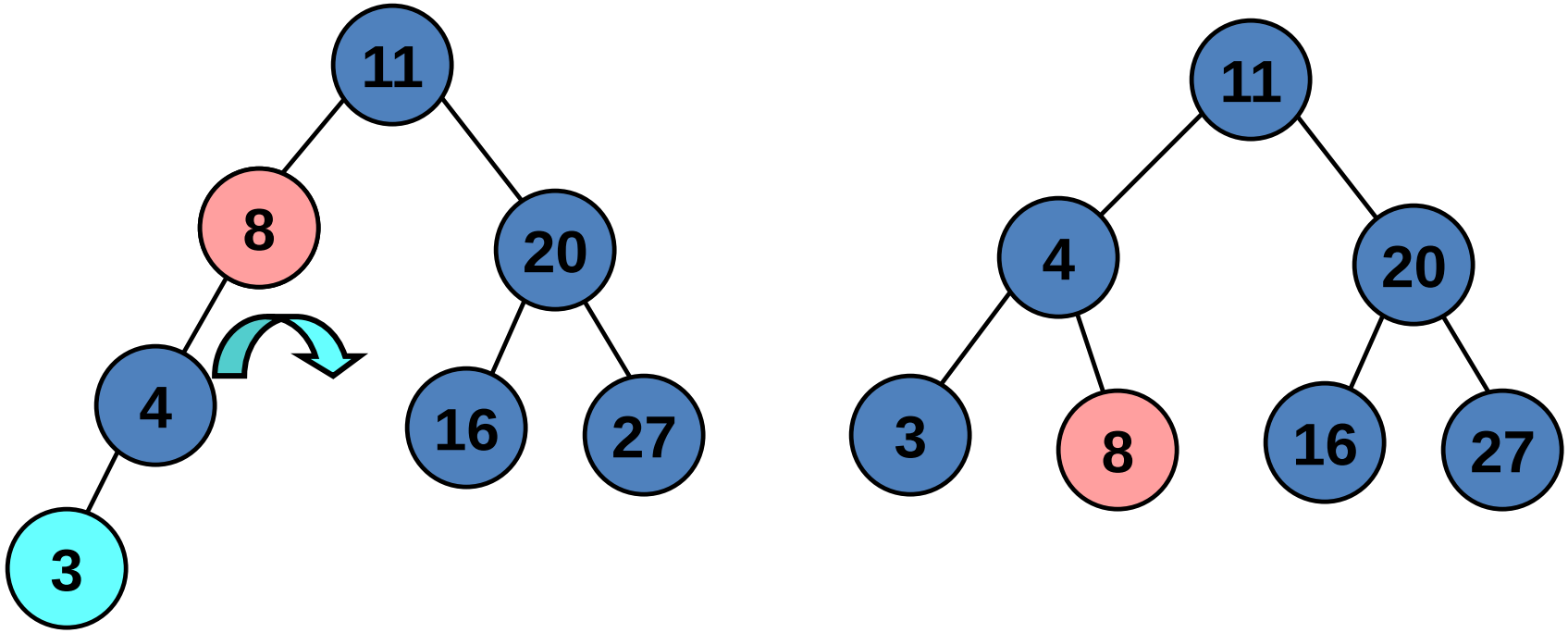
right rotation complete

Balance has been Restored!



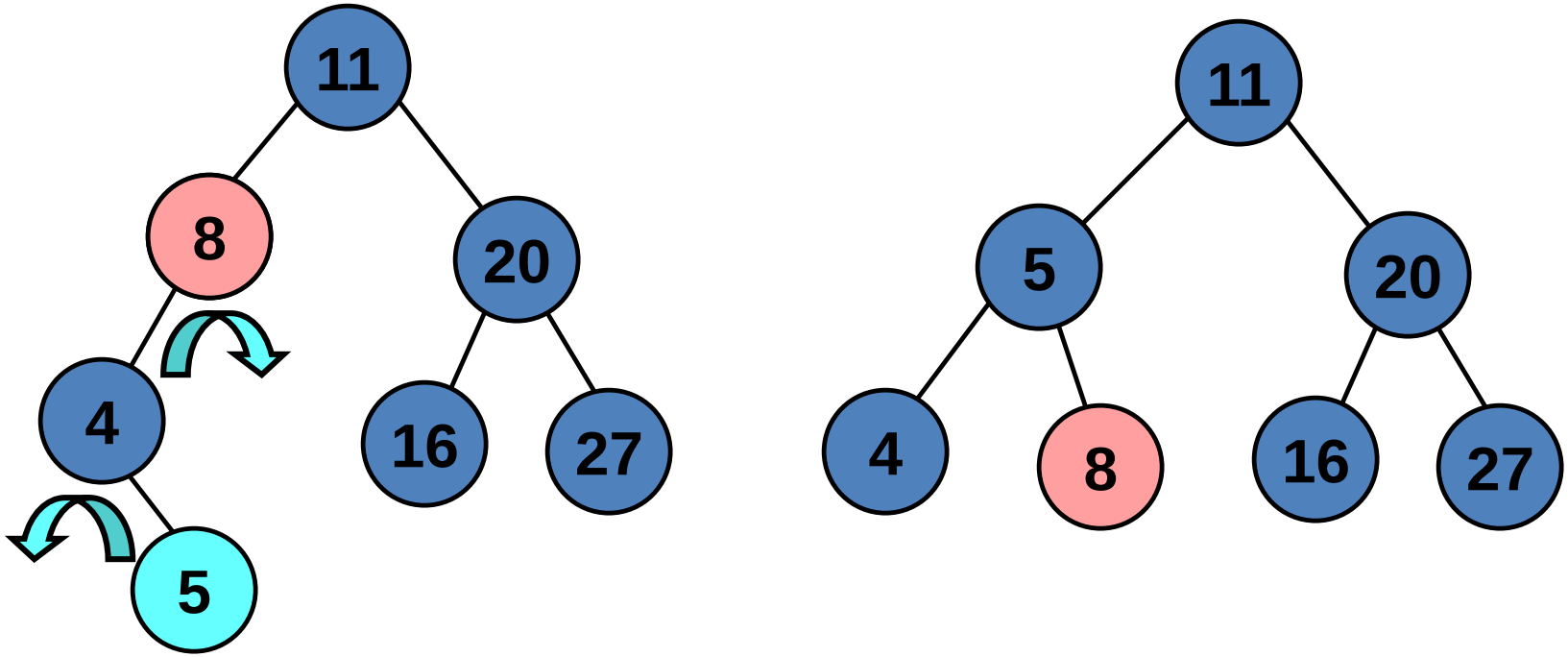
Example

Insert 3 into the AVL tree



Example

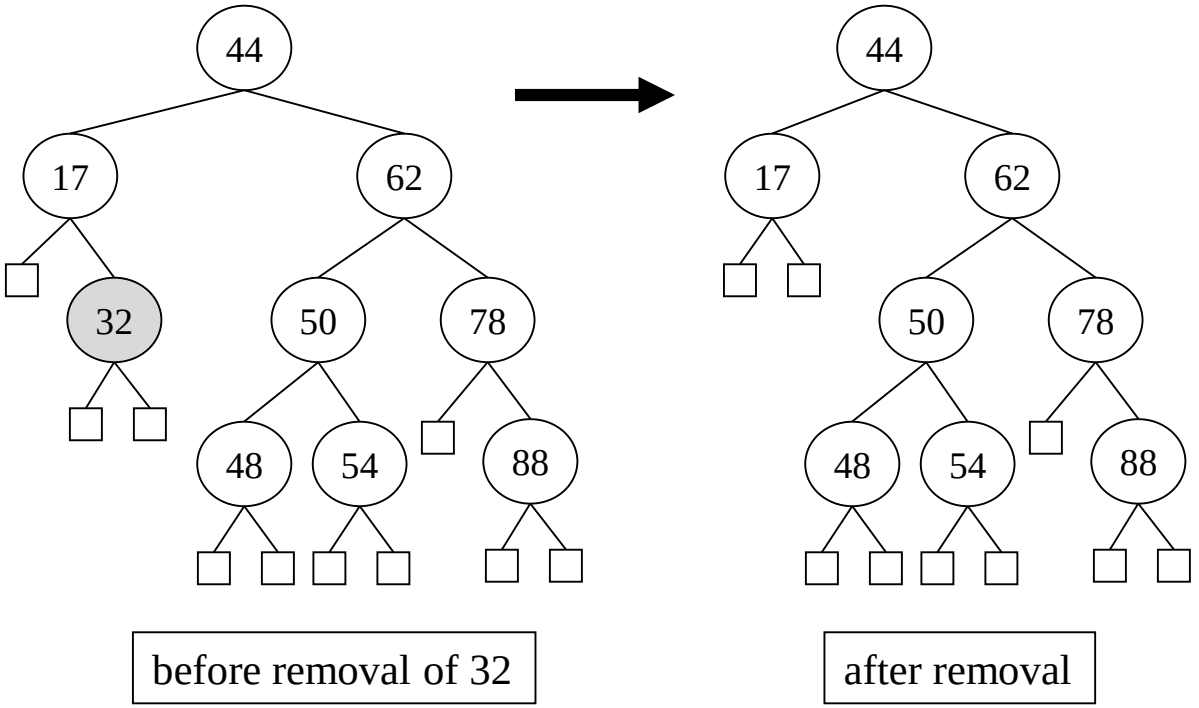
Insert 5 into the AVL tree



Removal from AVL Tree

Removal of key is done as in the binary search tree. The node removed will become an external node. After removal, its parent nodes may cause an imbalance.

Example:

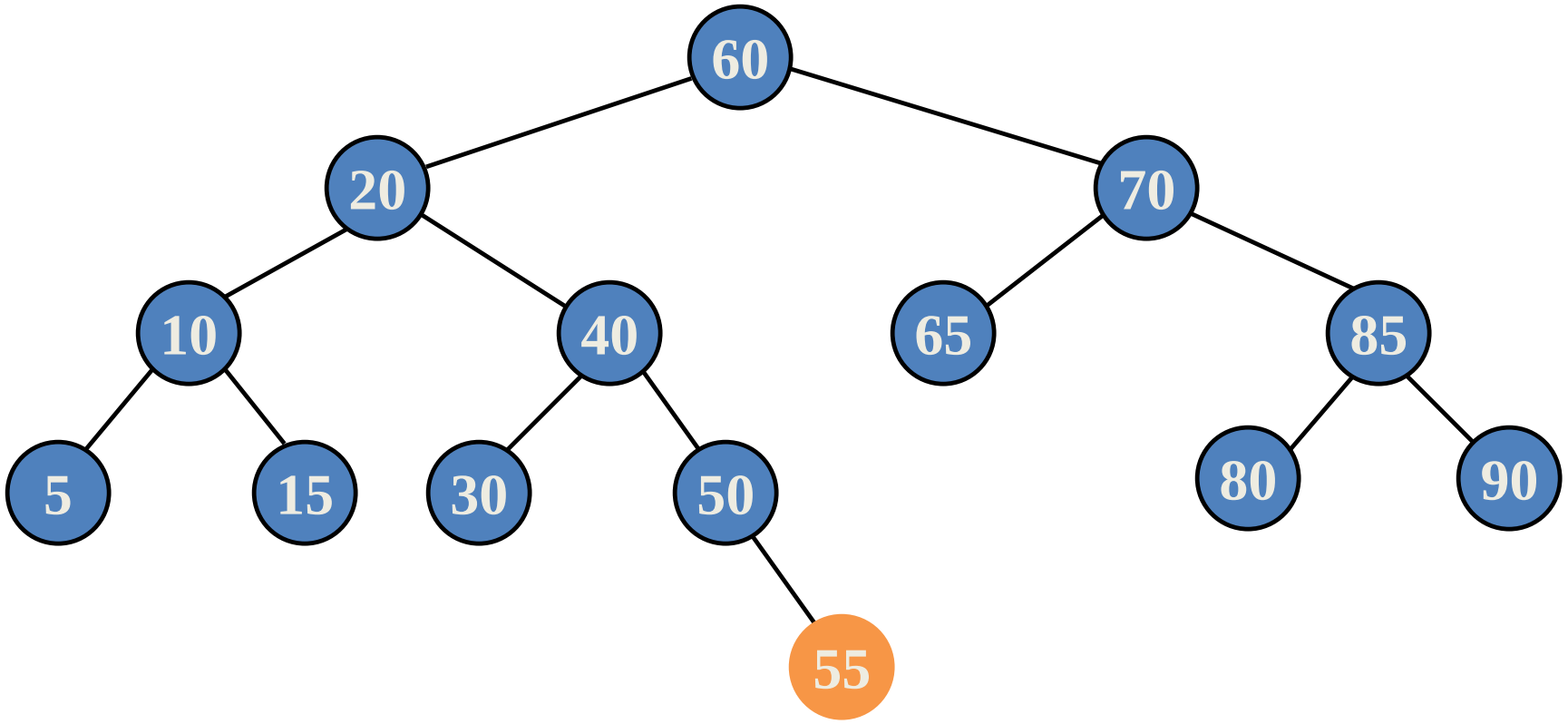


Remove P from AVL Trees

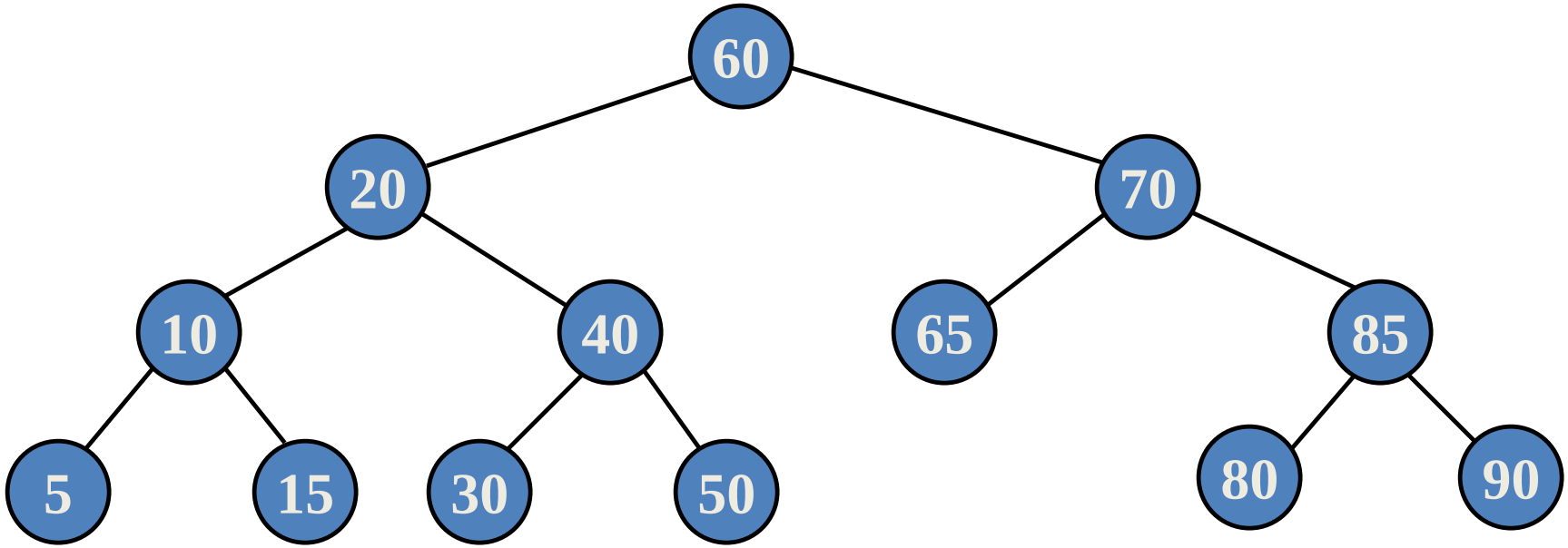
- Three cases arise:
  - Case 1: if P is a leaf, delete P
  - Case 2: if P has 1 child, use it to replace P
  - Case 3: if P has 2 children, replace P with its inorder predecessor

- Rebalance the tree, whenever needed.

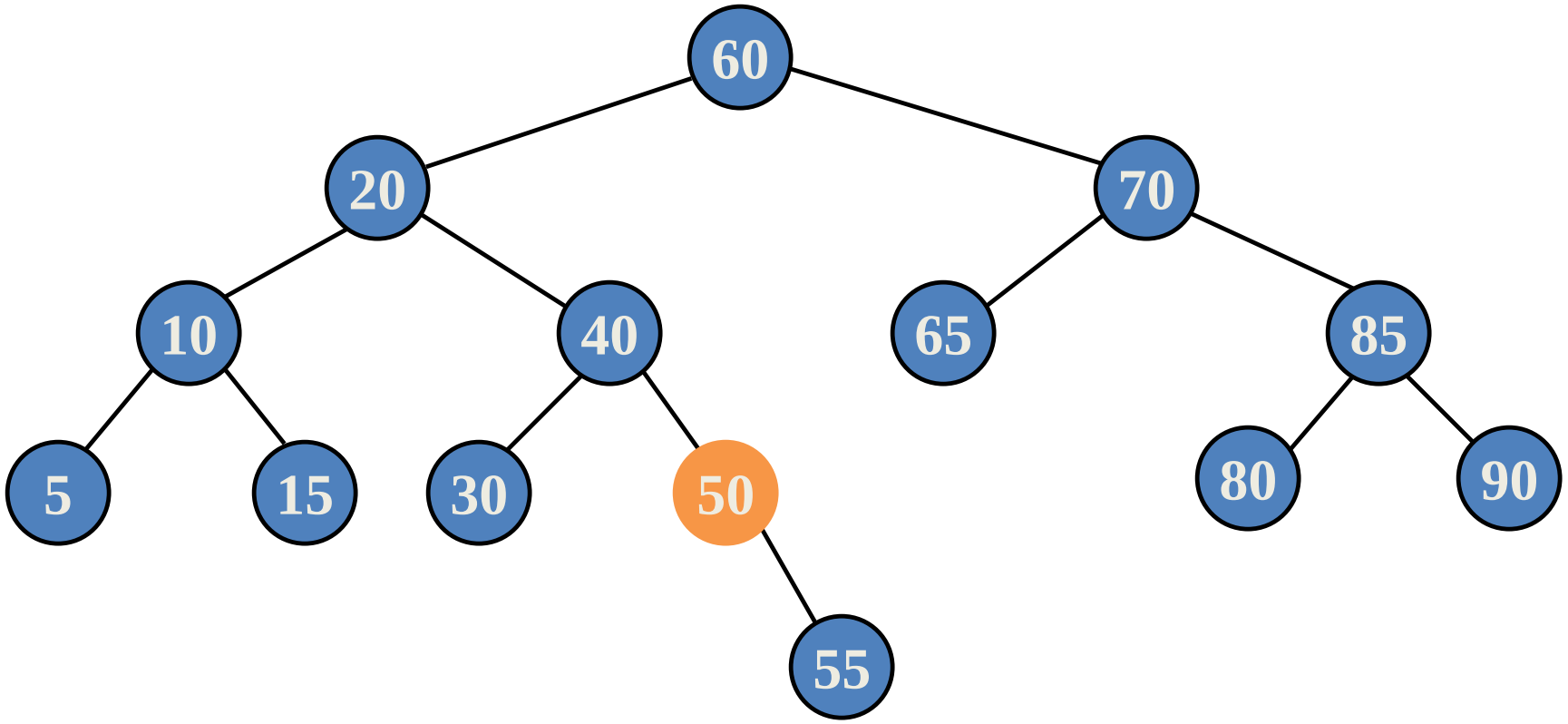
Delete 55 (case 1 : P is a leaf)



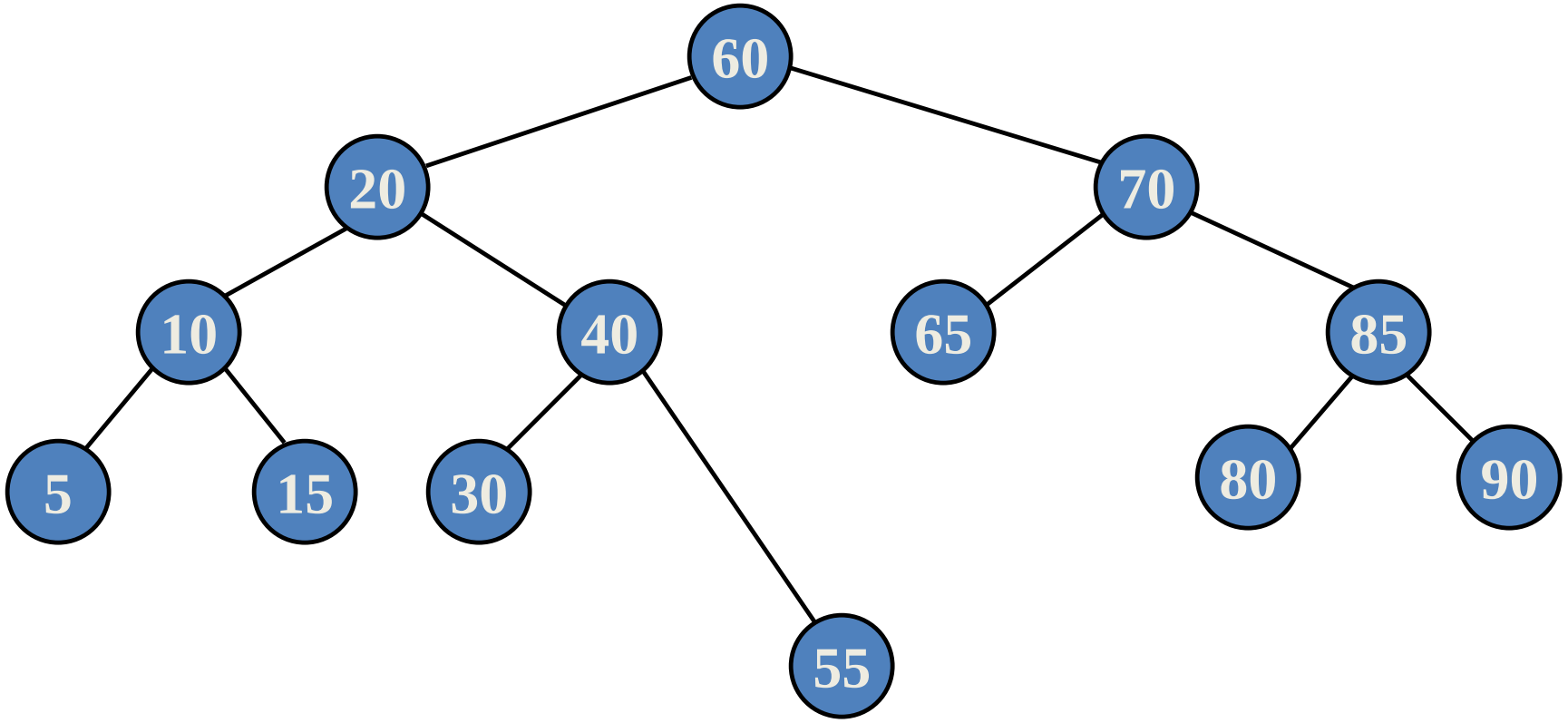
After deleting 55



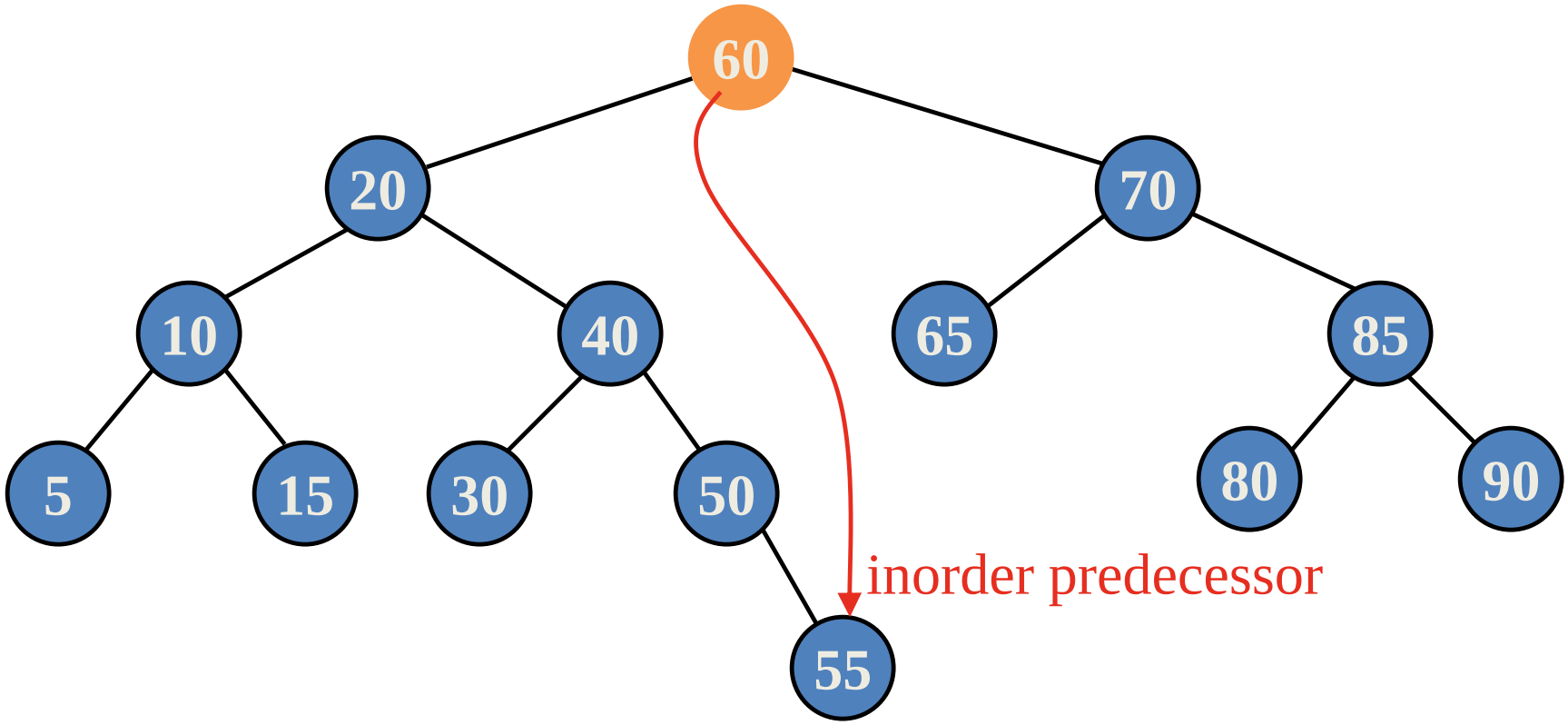
Delete 50 (case 2: P has 1 child)



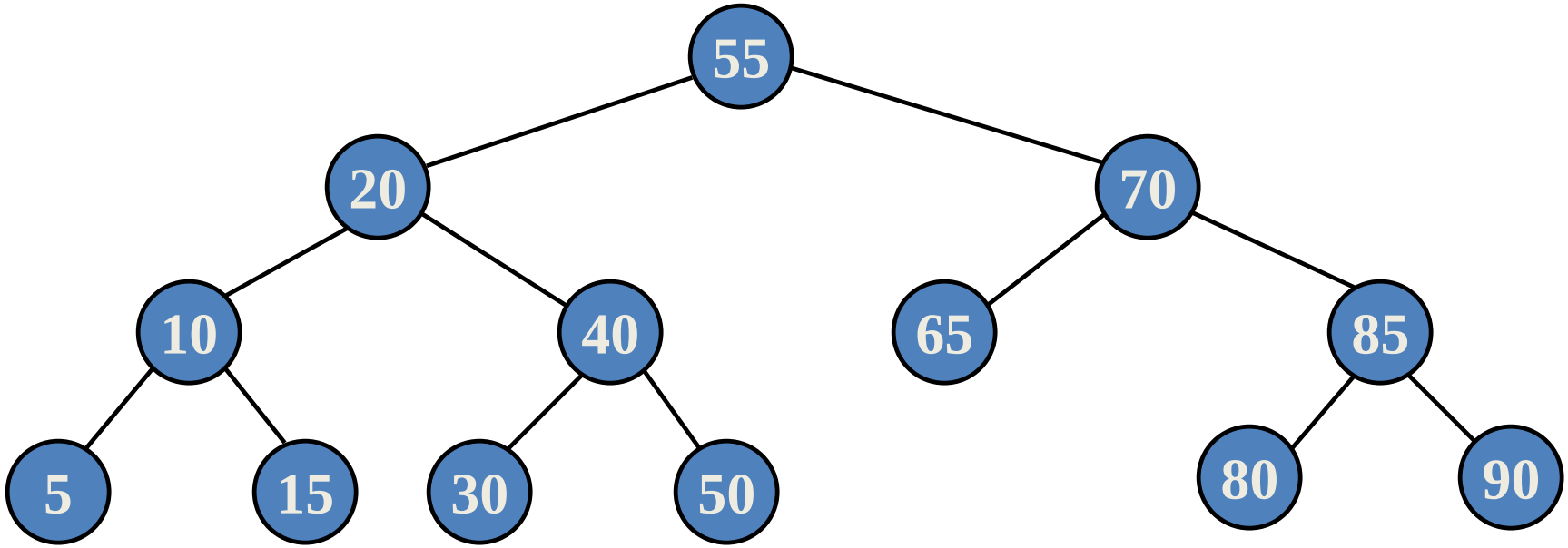
After deleting 50



Delete 60 (case 3: P has 2 children)



After deleting 60



Performance of AVL Trees

- find operation runs in $O(\log n)$
  - height of tree is $O(\log n)$
- insert running-time is $O(\log n)$
  - initial find is $O(\log n)$
  - maintaining heights is $O(\log n)$
- remove is $O(\log n)$
  - initial find is $O(\log n)$
  - maintaining heights is $O(\log n)$

Recommended Reading

- Goodrich *et al* – Ch. 9.2